

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2
Laboratorio



Proyecto 1
Codex Latinus

Jorge Anibal Bravo Rodríguez
202131782

Índice

Índice.....	2
Tecnologías Utilizadas.....	3
Diagrama de Clases.....	6
Arquitectura.....	7
Especificaciones de Palabras Reservadas, Símbolos y Gramática.....	8
1. Palabras Reservadas.....	8
1.1. Pig Latin (.pig).....	8
1.2. Lenguaje Y? (.y).....	8
1.3. Zetariano (.z).....	9
2. Gramáticas.....	10
2.1 Gramática Pig Latin.....	10
2.2 Y?.....	13
2.3 Zetariano.....	15
Tabla de Compatibilidad de Tipos.....	18
Jerarquía de tipos.....	19
1. Lenguaje Pig Latin (.pig).....	20
1.1. Tabla de Compatibilidad de Suma (+).....	20
1.2. Tabla de Compatibilidad de Operaciones Numéricas (-, *, /).....	20
1.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=).....	21
1.4. Tabla de Operaciones Lógicas (&&,).....	21
1.5. Tabla de Compatibilidad de Asignación (Destino = Origen).....	22
2. Lenguaje Y? (.y).....	23
2.1. Tabla de Compatibilidad de Suma (+).....	23
2.2. Tabla de Operaciones Numéricas (-, *, /).....	23
2.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=).....	24
2.4. Asignación (Destino = Origen).....	24
3. Lenguaje Zetariano (.z).....	25
3.1. Tabla Aritmética de Suma (+).....	25
3.2. Tabla de Operaciones Numéricas(-, *, /, %).....	25
3.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=).....	26
3.4. Asignación (Destino = Origen).....	26
3.5. Operador Ternario (cond ? RamaVerdadero : RamaFalso).....	27
4. Mapeo Unificado de Tipos a Código C Nativo.....	28

Tecnologías Utilizadas

El sistema compilador **3xtrat3rr3str3D** está diseñado bajo una arquitectura cliente-servidor que integra análisis léxico/sintáctico mediante ANTLR4, procesamiento semántico en Java y generación de código interpretable/ejecutable a bajo nivel.

Java

- **Descripción:** Lenguaje de programación orientado a objeto ampliamente utilizado en el desarrollo de software.
- **Versión:** 25
- **Uso:** Es el lenguaje principal del backend. Se utilizó para implementar todo el pipeline de compilación, incluyendo el análisis semántico, la construcción de los Árboles de Sintaxis Abstracta (AST), la generación de Código de Tres Direcciones (C3D) en cuartetas y todo lo relacionado al compilador.

ANTLR4

- **Descripción:** Herramienta que permite construir analizadores léxicos y sintácticos a partir de gramáticas.
- **Versión:** 4.13.2
- **Uso:** Se utilizó como motor de procesamiento léxico y sintáctico. A partir de los archivos de gramática .g4, genera los lexers y parsers para construir el árbol de derivación de los tres lenguajes soportados.

Javalin

- **Descripción:** Framework de desarrollo web REST ligero y flexible para Java enfocado en la simplicidad.
- **Versión:** 6.3.0
- **Uso:** Se utilizó para crear el servidor HTTP y exponer los endpoints de la API, permitiendo la comunicación entre la interfaz web y el motor de compilación en el puerto 7070.

Jackson

- **Descripción:** Biblioteca estándar de procesamiento JSON para Java, orientada al mapeo de objetos a datos en formato JSON y viceversa.
- **Versión:** 2.17.2
- **Uso:** Se utilizó para la serialización y deserialización de las respuestas del servidor hacia el cliente, enviando objetos como los errores de compilación, la tabla de símbolos, las cuartetas y las representaciones del AST.

GCC (GNU Compiler Collection)

- **Descripción:** Compilador de código abierto desarrollado por el proyecto GNU para compilar lenguajes como C o C++.
- **Versión:** Versión disponible instalada en el sistema operativo
- **Uso:** Se utilizó como el compilador final de bajo nivel. El backend invoca a gcc en segundo plano mediante comandos del sistema (gcc -Wall) para compilar el código C generado a partir de las cuartetas y producir un ejecutable nativo.

Svelte

- **Descripción:** Framework de interfaz de usuario web enfocado al desarrollo de páginas web.
- **Versión:** 5.56.8
- **Uso:** Se utilizó para el desarrollo del frontend de la aplicación web, construyendo la interfaz del IDE.

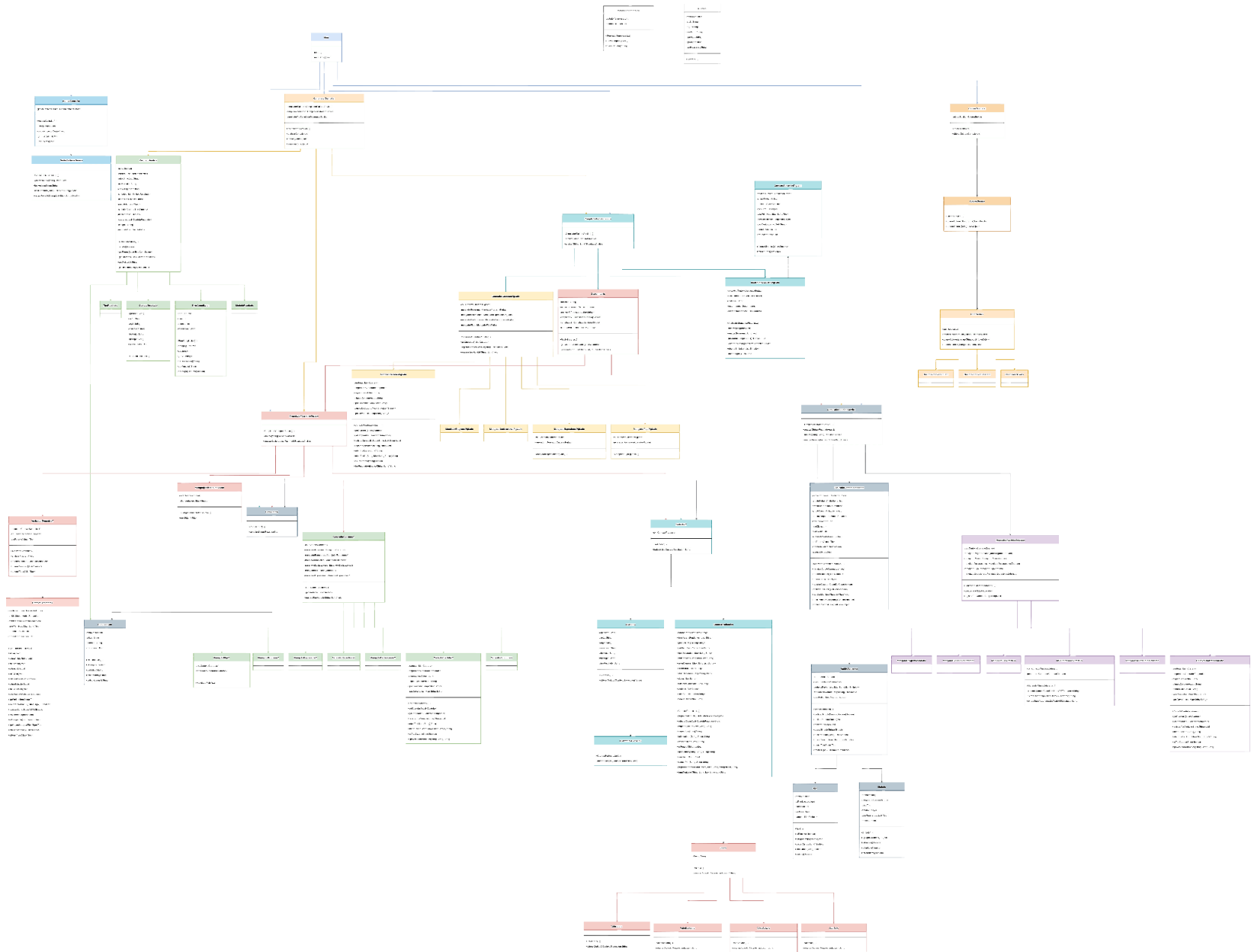
Monaco Editor

- **Descripción:** Editor de código web desarrollado por Microsoft que impulsa a Visual Studio Code.
- **Versión:** 0.56.0
- **Uso:** Se utilizó como el editor de texto central del IDE, permitiendo la edición de archivos de código fuente con soporte de resaltado de sintaxis personalizado para los lenguajes Pig Latin, Y? y Zetariano.

Bootstrap

- **Descripción:** Framework de diseño frontend responsivo enfocado en componentes CSS y componentes de interfaz prediseñados.
- **Versión:** 5.3.8
- **Uso:** Se utilizó para estilizar el frontend del IDE, incluyendo barras de navegación, botones, pestañas, modales de gestión de archivos y paneles flexibles.

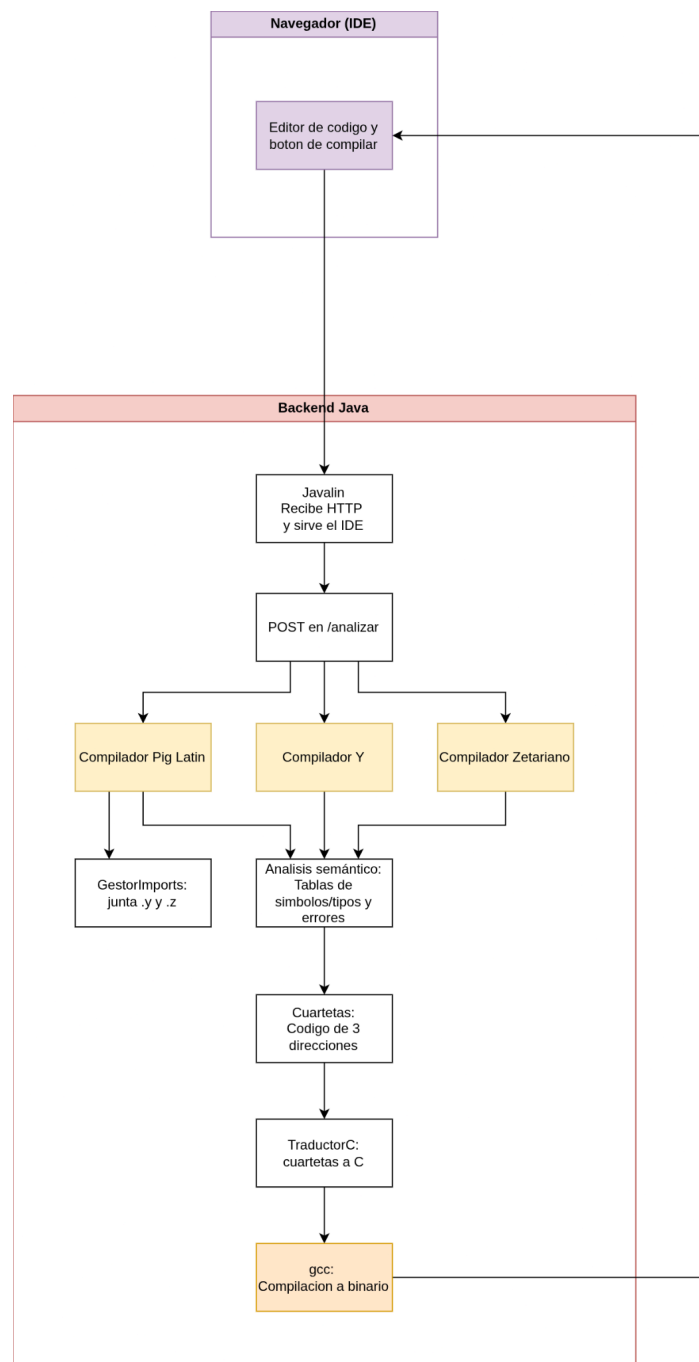
Diagrama de Clases



Arquitectura

El sistema compilador 3xtrat3rr3str3D está implementado en una arquitectura cliente-servidor desacoplada.

Se divide en un módulo de interfaz gráfica web (IDE) y un backend encargado de ejecutar el proceso completo de compilación, traducción a Código de Tres Direcciones (C3D) y compilación automática de C con GCC.



Especificaciones de Palabras Reservadas, Símbolos y Gramática

1. Palabras Reservadas

El compilador procesa tres lenguajes distintos de alto nivel, cada uno con especificaciones léxicas y tokens particulares:

1.1. Pig Latin (.pig)

Este es un lenguaje con tipos nombrados en latín, una sección obligatoria principal MAIOR cerrada por FINIS, sección global opcional VARIABILES y soporte exclusivo para importación de módulos externos con extensiones .y y .z.

- **Palabras Reservadas:** import, VARIABILES, MAIOR, FINIS, numerus, textum, decimalis, littera, bool, verum, falsus, esto, series, novus, si, aliter, dum, facere, per, interrompe, perge, finis, non.
- **Símbolos y Operadores:**
 - Operadores de Entrada y Salida: << (lectura), >> (impresión).
 - Aritméticos y Modificadores: +, -, *, /, ++, --.
 - Relacionales y Lógicos: ==, !=, >, <, >=, <=, &&, ||.
 - Asignación y Agrupación: =, [,], {, }, (,).
 - Delimitadores y Puntuación: ;, :, ,, ..
 - Comentarios: Inline // ..., Multilínea ## ... ##.

1.2. Lenguaje Y? (.y)

Lenguaje inspirado en python orientado a funciones y estructuras, estructurado en secciones %estructuras y %funciones. Utiliza indentación como delimitador mediante los tokens INDENT y DEDENT.

- **Palabras Reservadas:** %estructuras, %funciones, entero, cadena, flotante, caracter, booleano, verdadero, falso, estructura, definir, retornar, si, entonces, sino, contrario, elegir, caso, siempre, romper, continuar, para, mientras, hacer.
- **Símbolos y Operadores:**
 - Operadores y Flechas: -> (indicador de tipo de retorno en funciones).
 - Aritméticos y Modificadores: +, -, *, /, ++, --.
 - Relacionales, Lógicos y Negación: ==, !=, >, <, >=, <=, &&, ||, !.
 - Asignación y Agrupación: =, [,], {, }, (,).
 - Delimitadores y Puntuación: :, ,, . (Nota: El ; está reservado exclusivamente para la cabecera de ciclos para).
 - Especiales de Bloque: NEWLINE, INDENT, DEDENT.

1.3. Zetariano (.z)

Lenguaje orientado a objetos con sintaxis inspirada en Java, con clases, métodos, constructores, atributos y palabras reservadas en inglés.

- **Palabras Reservadas:** public, class, void, new, null, int, double, char, boolean, String, true, false, if, else, switch, case, default, for, while, do, break, continue, return.
- **Símbolos y Operadores:**
 - Operadores Únicos: % (módulo), ? y : (operador ternario cond ? v1 : v2).
 - Asignaciones Compuestas: +=, -=, *=.
 - Aritméticos y Modificadores: +, -, *, /, ++, --.
 - Relacionales, Lógicos y Unarios: ==, !=, >, <, >=, <=, &&, ||, !, =.
 - Agrupación y Delimitadores: [,], {, }, (,), ;, ,, ..

2. Gramáticas

2.1 Gramática Pig Latin

```
grammar PigLatin;

//=====
// - - - - - REGLAS SINTACTICAS (Parser) - - - - -
//=====

// =====
// ESTRUCTURA PRINCIPAL
// =====

programa
: seccion_importaciones?
  seccion_global_variables?
  seccion_maior
  EOF
;

seccion_importaciones
: (IMPORT ruta_importacion)*
;

ruta_importacion
: IDENTIFICADOR (PUNTO IDENTIFICADOR)*
;

seccion_global_variables
: VARIABLES MAYOR_QUE declaracion_variable*
;

seccion_maior
: MAIOR MAYOR_QUE instruccion_flujo* FINIS PUNTO_Y_COMA
;

// =====
// TIPOS DE DATOS
// =====

tipo_dato
: NUMERUS
| TEXTUM
| DECIMALIS
| LITTERA
| BOOL
| IDENTIFICADOR
;

// =====
// VALORES ASIGNABLES
// =====

variable_asignable
: IDENTIFICADOR # valorAsignableSimple
| variable_asignable CORCHETE_IZQ expresion CORCHETE_DER # valorAsignableArray
| variable_asignable PUNTO IDENTIFICADOR # valorAsignableMiembroEstructura
;

// =====
// EXPRESIONES Y VALORES
// =====

expresion
: PAR_IZQ expresion PAR_DER # exprParentesis
| NOVUS IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER # exprInstanciaObjeto
| IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER # exprLlamadaFuncion
| expresion PUNTO IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER # exprLlamadaMetodo
| expresion CORCHETE_IZQ expresion CORCHETE_DER # exprAccesoPosicionArray
| expresion PUNTO IDENTIFICADOR # exprAccesoMiembroEstructura
| variable_asignable INCREMENTO # exprPostIncremento
| variable_asignable DECREMENTO # exprPostDecremento
| LLAVE_IZQ lista_expresiones LLAVE_DER # exprListaLiteral
| RESTA expresion # exprNegativa
| NON expresion # exprNegada
| INCREMENTO variable_asignable # exprPreIncremento
| DECREMENTO variable_asignable # exprPreDecremento
| expresion ( MULT | DIV ) expresion # exprMultiplicacionDivision
| expresion ( SUMA | RESTA ) expresion # exprSumaResta
| expresion ( IGUAL_QUE | DIFERENTE_QUE
| MAYOR_QUE | MENOR_QUE
| MAYOR_IGUAL_QUE | MENOR_IGUAL_QUE ) expresion # exprRelacional
| expresion AND expresion # exprAnd
| expresion OR expresion # exprOr
| valor_primitivo # exprPrimitivo
;
```

```

valor_primitivo
: ENTERO
| DECIMAL
| CADENA
| CARACTER
| VERUM
| FALSUS
| IDENTIFICADOR
;

lista_expresiones
: expresion (COMA expresion)*
;

lista_atributos_instancia
: atributo_instancia (COMA atributo_instancia)* COMA?
;

atributo_instancia
: IDENTIFICADOR DOS_PUNTOS expresion # campoConNombre
| expresion # campoPosicional
;

// =====
// DECLARACION DE VARIABLES
// =====

declaracion_variable
: ESTO IDENTIFICADOR DOS_PUNTOS NOVUS IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER PUNTO_Y_COMA
# declObjetoNovus
| ESTO IDENTIFICADOR DOS_PUNTOS IDENTIFICADOR LLAVE_IZQ lista_atributos_instancia LLAVE_DER PUNTO_Y_COMA
# declEstructuraConValores
| ESTO IDENTIFICADOR DOS_PUNTOS tipo_dato (ASIGNACION? expresion)? PUNTO_Y_COMA
# declConTipoYValor
| ESTO IDENTIFICADOR DOS_PUNTOS (VERUM | FALSUS) PUNTO_Y_COMA
# declBooleanaImplicita
| SERIES IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS tipo_dato PUNTO_Y_COMA
# declArraySinDatos
| SERIES IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS tipo_dato LLAVE_IZQ lista_expresiones
LLAVE_DER PUNTO_Y_COMA # declArrayConDatos
| SERIES IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS IDENTIFICADOR PUNTO_Y_COMA
# declArrayEstructura
| SERIES IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS
tipo_dato PUNTO_Y_COMA # declMatrizSinDatos
| SERIES IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS
tipo_dato LLAVE_IZQ fila_matriz_pig (COMA fila_matriz_pig)* LLAVE_DER PUNTO_Y_COMA # declMatrizConDatos
;

fila_matriz_pig
: LLAVE_IZQ lista_expresiones LLAVE_DER
;

// =====
// ASIGNACION
// =====

asignacion
: variable_asignable ASIGNACION expresion PUNTO_Y_COMA #
asignacionGeneral
;

// =====
// INSTRUCCIONES DE FLUJO
// =====

instruccion_flujo
: asignacion # stmtAsignacion
| condicional # stmtCondicional
| ciclo # stmtCiclo
| instruccion_lectura # stmtLectura
| instruccion_impresion # stmtImpresion
| INTERRUMPE PUNTO_Y_COMA # stmtInterrumpe
| PERGE PUNTO_Y_COMA # stmtPerge
| expresion PUNTO_Y_COMA # stmtExpresion
;

bloque
: LLAVE_IZQ instruccion_flujo* LLAVE_DER
;

// =====
// CONDICIONALES Y CICLOS
// =====

condicional
: SI PAR_IZQ expresion PAR_DER bloque
(ALITER PAR_IZQ expresion PAR_DER bloque)*
(ALITER bloque)?
FINIS_BLOQUE PUNTO_Y_COMA # statementSi
;

```

```

ciclo
:      DUM      PAR_IZQ      expresion      PAR_DER      bloque      FINIS_BLOQUE      PUNTO_Y_COMA
# cicloDum      |      FACERE      bloque      DUM      PAR_IZQ      expresion      PAR_DER      PUNTO_Y_COMA
# cicloFacere      | PER PAR_IZQ init_per PUNTO_Y_COMA expresion PUNTO_Y_COMA paso_per PAR_DER bloque (FINIS_BLOQUE
PUNTO_Y_COMA)? # cicloPer
;

init_per
: ESTO IDENTIFICADOR DOS_PUNTOS tipo_dato ASIGNACION? expresion # initPerDecl
| variable_asignable ASIGNACION expresion # initPerAsig
;

paso_per
: expresion # pasoPerExpr
| variable_asignable ASIGNACION expresion # pasoPerAsig
;

// =====
// INSTRUCCIONES ESPECIALES (ENTRADA / SALIDA)
// =====

instruccion_lectura
: LEER PUNTO_Y_COMA? #
lecturaConsolaSimple
| variable_asignable LEER PUNTO_Y_COMA? #
lecturaConsolaAVariable
;

instruccion_impresion
: IMPRIMIR elemento_imprimir (IMPRIMIR elemento_imprimir)* PUNTO_Y_COMA? # impresionEncadenada
;

elemento_imprimir
: expresion
;

```

2.2 Y?

```
grammar YLenguaje;

tokens { INDENT, DEDENT }

// =====
// - - - - - REGLAS SINTACTICAS (Parser) - - - - -
// =====

// =====
// ESTRUCTURA PRINCIPAL
// =====

programa
    : seccion_estructuras?
      seccion_funciones
      EOF
    ;

seccion_estructuras
    : ESTRUCTURAS_TAG (definicion_struct | NEWLINE)*
    ;

seccion_funciones
    : FUNCIONES_TAG (definicion_funcion | NEWLINE)*
    ;

// =====
// TIPOS DE DATOS
// =====

tipo_dato
    : TIPO_ENTERO
    | TIPO_CADENA
    | TIPO_FLOTANTE
    | TIPO_CARACTER
    | TIPO_BOOLEANO
    | IDENTIFICADOR
    ;

// =====
// DEFINICION DE ESTRUCTURAS
// =====

definicion_struct
    : ESTRUCTURA IDENTIFICADOR DOS_PUNTOS NEWLINE INDENT atributo_struct+ DEDENT      # defEstructura
    ;

atributo_struct
    : tipo_dato IDENTIFICADOR NEWLINE                                     # atributoSimple
    | tipo_dato IDENTIFICADOR CORCHETE_IZQ NUMERO_ENTERO CORCHETE_DER NEWLINE      # atributoArray
    ;

// =====
// DEFINICION DE FUNCIONES
// =====

definicion_funcion
    : DEFINIR IDENTIFICADOR PAR_IZQ parametros? PAR_DER DOS_PUNTOS cuerpo_funcion      #
defFuncionSinRetorno
    | DEFINIR IDENTIFICADOR PAR_IZQ parametros? PAR_DER FLECHA tipo_dato DOS_PUNTOS cuerpo_funcion      #
defFuncionConRetorno
    ;

parametros
    : parametro (COMA parametro)*
    ;

parametro
    : tipo_dato IDENTIFICADOR                                     # paramSimple
    | CORCHETE_IZQ CORCHETE_DER tipo_dato IDENTIFICADOR        # paramArray
    | LLAVE_IZQ LLAVE_DER IDENTIFICADOR IDENTIFICADOR          # paramEstructura
    ;

cuerpo_funcion
    : NEWLINE INDENT instruccion+ NEWLINE? DEDENT
    ;

// =====
// INSTRUCCIONES
// =====

instruccion
    : declaracion_variable terminador                                # stmtDeclaracion
```

```

| asignacion terminador          # stmtAsignacion
| definicion_struct              # stmtEstructuralLocal
| condicional                    # stmtCondicional
| seleccion                      # stmtSeleccion
| ciclo                          # stmtCiclo
| retorno terminador            # stmtRetorno
| CONTINUAR terminador          # stmtContinuar
| ROMPER terminador             # stmtRomper
| expresion terminador          # stmtExpresion
;

terminador
: NEWLINE
;

retorno
: RETORNAR expresion
;

// =====
// DECLARACION DE VARIABLES
// =====

declaracion_variable
: tipo_dato IDENTIFICADOR (ASIGNACION expresion)? #
declConTipoYValor
| tipo_dato IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER #
declArraySinValores
| tipo_dato IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER ASIGNACION LLAVE_IZQ lista_expresiones LLAVE_DER #
declArrayConValores
| tipo_dato IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER CORCHETE_IZQ expresion CORCHETE_DER # declMatriz
| tipo_dato IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER CORCHETE_IZQ expresion CORCHETE_DER ASIGNACION LLAVE_IZQ
fila_matriz (COMA fila_matriz)* LLAVE_DER # declMatrizConValores
;

fila_matriz
: LLAVE_IZQ lista_expresiones LLAVE_DER
;

// =====
// ASIGNACION
// =====

asignacion
: variable_asignable ASIGNACION expresion
;

variable_asignable
: IDENTIFICADOR # varSimple
| variable_asignable CORCHETE_IZQ expresion CORCHETE_DER # varArray
| variable_asignable PUNTO IDENTIFICADOR # varMiembro
;

// =====
// CONDICIONALES
// =====

condicional
: SI PAR_IZQ expresion PAR_DER ENTONCES bloque
(SINO PAR_IZQ expresion PAR_DER ENTONCES bloque)*
(CONTRARIO bloque)? # statementSi
;

bloque
: NEWLINE INDENT instruccion+ NEWLINE? DEDENT
| instruccion
;

seleccion
: ELEGIR PAR_IZQ expresion PAR_DER DOS_PUNTOS NEWLINE INDENT caso_seleccion+ caso_defecto? DEDENT # statementElegir
;

caso_seleccion
: CASO valor_primitivo DOS_PUNTOS NEWLINE INDENT instruccion+ DEDENT
;

caso_defecto
: SIEMPRE DOS_PUNTOS NEWLINE INDENT instruccion+ DEDENT
;

// =====
// CICLOS
// =====

ciclo
: PARA PAR_IZQ init_para PUNTO_Y_COMA expresion PUNTO_Y_COMA paso_para PAR_DER DOS_PUNTOS bloque # cicloPara
| MIENTRAS PAR_IZQ expresion PAR_DER HACER bloque # cicloMientras

```

```

| HACER DOS_PUNTOS bloque MIENTRAS PAR_IZQ expresion PAR_DER NEWLINE # cicloHacer
;

init_para
: tipo_dato IDENTIFICADOR ASIGNACION expresion # initParaDecl
| variable_asignable ASIGNACION expresion # initParaAsig
;

paso_para
: expresion # pasoParaExpr
| variable_asignable ASIGNACION expresion # pasoParaAsig
;

// =====
// EXPRESIONES Y VALORES
// =====

expresion
: PAR_IZQ expresion PAR_DER # exprParentesis
| IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER # exprLlamadaFuncion
| expresion CORCHETE_IZQ expresion CORCHETE_DER # exprAccesoArray
| expresion PUNTO IDENTIFICADOR # exprAccesoMiembro
| variable_asignable INCREMENTO # exprPostIncremento
| variable_asignable DECREMENTO # exprPostDecremento
| LLAVE_IZQ lista_expresiones LLAVE_DER # exprListaliteral
| RESTA expresion # exprNegativa
| NEGACION expresion # exprNegada
| expresion (MULT | DIV) expresion # exprMultiplicacionDivision
| expresion (SUMA | RESTA) expresion # exprSumaResta
| expresion (IGUAL_QUE | DIFERENTE_QUE | MAYOR_QUE | MENOR_QUE | MAYOR_IGUAL | MENOR_IGUAL) expresion # exprRelacional
| expresion AND expresion # exprAnd
| expresion OR expresion # exprOr
| valor_primitivo # exprPrimitivo
;

valor_primitivo
: NUMERO_ENTERO
| NUMERO_DECIMAL
| CADENA
| CARACTER
| VERDADERO
| FALSO
| IDENTIFICADOR
;

lista_expresiones
: expresion (COMA expresion)*
;

```

2.3 Zetariano

```
grammar Zetariano;

// =====
// - - - - - REGLAS SINTACTICAS (Parser) - - - - -
// =====

// =====
// ESTRUCTURA PRINCIPAL
// =====

programa
    : definicion_clase EOF
    ;

definicion_clase
    : PUBLIC CLASS IDENTIFICADOR LLAVE_IZQ miembro_clase* LLAVE_DER      # defClase
    ;

miembro_clase
    : atributo_clase              # miembroAtributo
    | constructor                 # miembroConstructor
    | metodo                      # miembroMetodo
    ;

// =====
// TIPOS DE DATOS
// =====

tipo_dato
    : INT
    | DOUBLE
    | CHAR
    | BOOLEAN
    | STRING
    | IDENTIFICADOR
    ;

// =====
// ATRIBUTOS DE LA CLASE
// =====

atributo_clase
    : PUBLIC? tipo_dato IDENTIFICADOR PUNTO_Y_COMA                # atributoSimple
    | PUBLIC? tipo_dato (CORCHETE_IZQ CORCHETE_DER)+ IDENTIFICADOR PUNTO_Y_COMA  # atributoArray
    ;

// =====
// CONSTRUCTORES Y METODOS
// =====

constructor
    : PUBLIC IDENTIFICADOR PAR_IZQ parametros? PAR_DER LLAVE_IZQ instruccion* LLAVE_DER      #
defConstructor
    ;

metodo
    : PUBLIC VOID IDENTIFICADOR PAR_IZQ parametros? PAR_DER LLAVE_IZQ instruccion* LLAVE_DER      #
metodoSinRetorno
    | PUBLIC tipo_dato IDENTIFICADOR PAR_IZQ parametros? PAR_DER LLAVE_IZQ instruccion* LLAVE_DER      #
metodoConRetorno
    ;

parametros
    : parametro (COMA parametro)*
    ;

parametro
    : tipo_dato IDENTIFICADOR                # paramSimple
    | tipo_dato (CORCHETE_IZQ CORCHETE_DER)+ IDENTIFICADOR      # paramArray
    ;

// =====
// INSTRUCCIONES
// =====
```



```

instruccion
: declaracion_variable          # stmtDeclaracion
| asignacion                    # stmtAsignacion
| condicional                   # stmtCondicional
| seleccion                     # stmtSeleccion
| ciclo                         # stmtCiclo
| RETURN expresion? PUNTO_Y_COMA # stmtReturn
| BREAK PUNTO_Y_COMA           # stmtBreak
| CONTINUE PUNTO_Y_COMA        # stmtContinue
| expresion PUNTO_Y_COMA       # stmtExpresion
;

// =====
// DECLARACION DE VARIABLES
// =====

declaracion_variable
: tipo_dato (CORCHETE_IZQ CORCHETE_DER)* IDENTIFICADOR (ASIGNACION expresion)? PUNTO_Y_COMA
# declConTipo
| tipo_dato (CORCHETE_IZQ CORCHETE_DER)+ IDENTIFICADOR ASIGNACION LLAVE_IZQ lista_expresiones
LLAVE_DER PUNTO_Y_COMA # declConListaLiteral
| tipo_dato (CORCHETE_IZQ CORCHETE_DER)+ IDENTIFICADOR ASIGNACION LLAVE_IZQ init_matriz LLAVE_DER
PUNTO_Y_COMA # declConMatrizLiteral
;

init_matriz
: init_elemento (COMA init_elemento)*
;

init_elemento
: expresion
| LLAVE_IZQ init_matriz LLAVE_DER
;

// =====
// ASIGNACION
// =====

asignacion
: variable_asignable ASIGNACION expresion PUNTO_Y_COMA #
asignacionSimple
| variable_asignable (MAS_IGUAL | MENOS_IGUAL | MULT_IGUAL) expresion PUNTO_Y_COMA #
asignacionCompuesta
;

variable_asignable
: IDENTIFICADOR # varSimple
| variable_asignable CORCHETE_IZQ expresion CORCHETE_DER # varArray
| variable_asignable PUNTO IDENTIFICADOR # varMiembro
;

// =====
// CONDICIONALES
// =====

condicional
: IF PAR_IZQ expresion PAR_DER bloque
(ELSE IF PAR_IZQ expresion PAR_DER bloque)*
(ELSE bloque)? # statementIf
;

bloque
: LLAVE_IZQ instruccion* LLAVE_DER
| instruccion
;

seleccion
: SWITCH PAR_IZQ expresion PAR_DER LLAVE_IZQ caso_switch* caso_default? LLAVE_DER #
statementSwitch
;

caso_switch
: CASE valor_primitivo DOS_PUNTOS instruccion*
;

caso_default
: DEFAULT DOS_PUNTOS instruccion*
;

```

```

// =====
// CICLOS
// =====

ciclo
: FOR PAR_IZQ init_for? PUNTO_Y_COMA expresion? PUNTO_Y_COMA paso_for? PAR_DER bloque #
cicloFor
| WHILE PAR_IZQ expresion PAR_DER bloque #
cicloWhile
| DO bloque WHILE PAR_IZQ expresion PAR_DER PUNTO_Y_COMA #
cicloDoWhile
;

init_for
: tipo_dato IDENTIFICADOR ASIGNACION expresion # initForDecl
| variable_asignable ASIGNACION expresion # initForAsig
;

paso_for
: expresion # pasoForExpr
| variable_asignable ASIGNACION expresion # pasoForAsig
;

// =====
// EXPRESIONES Y VALORES
// =====

expresion
: PAR_IZQ expresion PAR_DER # exprParentesis
| NEW IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER #
exprInstanciaObjeto
| NEW tipo_dato (CORCHETE_IZQ expresion CORCHETE_DER)+ #
exprInstanciaArreglo
| IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER #
exprLlamadaFuncion
| expresion PUNTO IDENTIFICADOR PAR_IZQ lista_expresiones? PAR_DER #
exprLlamadaMetodo
| expresion CORCHETE_IZQ expresion CORCHETE_DER # exprAccesoArray
| expresion PUNTO IDENTIFICADOR #
exprAccesoMiembro
| variable_asignable INCREMENTO #
exprPostIncremento
| variable_asignable DECREMENTO #
exprPostDecremento
| RESTA expresion # exprNegativa
| NEGACION expresion # exprNegada
| expresion (MULT | DIV | MOD) expresion #
exprMultiplicacionDivisionModulo
| expresion (SUMA | RESTA) expresion # exprSumaResta
| expresion (IGUAL_QUE | DIFERENTE_QUE | MAYOR_QUE | MENOR_QUE
| MAYOR_IGUAL_QUE | MENOR_IGUAL_QUE) expresion # exprRelacional
| expresion AND expresion # exprAnd
| expresion OR expresion # exprOr
| expresion INTERROGACION expresion DOS_PUNTOS expresion # exprTernario
| valor_primitivo # exprPrimitivo
;

valor_primitivo
: NUMERO_ENTERO
| NUMERO_DECIMAL
| CADENA
| CARACTER
| TRUE
| FALSE
| NULL
| IDENTIFICADOR
;

lista_expresiones
: expresion (COMA expresion)*
;

```

Tabla de Compatibilidad de Tipos

Esta sección detalla las reglas de comprobación de tipos, compatibilidad implícita, compatibilidad de operaciones y el mapeo final hacia el código C ejecutable para los tres lenguajes: Pig Latin, Y? y Zetariano.

Jerarquía de tipos

Los tres lenguajes tienen una jerarquía de tipos definida en el documento de la práctica, donde se tiene la siguiente prioridad:

Tipo	Prioridad
texto	5
número decimal	4
número entero	3
caracter	2
booleano	1

Convenciones

- Filas: Tipo operando izquierdo (Izq) o Tipo destino.
- Columnas: Tipo operando derecho (Der) o Tipo origen.
- Nombre de tipo: Tipo resultante de la expresión evaluada.
- ERROR: Violación semántica que detiene la generación de C3D.
- OK: Asignación directa permitida.
- OK (conversión a tipo de mayor jerarquía): Asignación permitida mediante conversión implícita.

1. Lenguaje Pig Latin (.pig)

Tipos Primitivos: numerus, decimalis, textum, littera, bool.

1.1. Tabla de Compatibilidad de Suma (+)

El operador + realiza suma numérica o concatenación si al menos uno de los operandos es de tipo textum.

Izq / Der	numerus	decimalis	textum	littera	bool
numerus	numerus	decimalis	textum	ERROR	ERROR
decimalis	decimalis	decimalis	textum	ERROR	ERROR
textum	textum	textum	textum	textum	textum
littera	ERROR	ERROR	textum	ERROR	ERROR
bool	ERROR	ERROR	textum	ERROR	ERROR

1.2. Tabla de Compatibilidad de Operaciones Numéricas (-, *, /)

Aplica exclusivamente entre tipos numéricos (numerus y decimalis).

Izq / Der	numerus	decimalis	textum	littera	bool
numerus	numerus	decimalis	ERROR	ERROR	ERROR
decimalis	decimalis	decimalis	ERROR	ERROR	ERROR
textum	ERROR	ERROR	ERROR	ERROR	ERROR
littera	ERROR	ERROR	ERROR	ERROR	ERROR
bool	ERROR	ERROR	ERROR	ERROR	ERROR

1.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=)

Devuelve un tipo bool siempre que los operandos sean del mismo tipo de dato.

Izq / Der	numerus	decimalis	textum	littera	bool
numerus	bool	bool	ERROR	ERROR	ERROR
decimalis	bool	bool	ERROR	ERROR	ERROR
textum	ERROR	ERROR	bool	ERROR	ERROR
littera	ERROR	ERROR	ERROR	ERROR	ERROR
bool	ERROR	ERROR	ERROR	ERROR	bool

1.4. Tabla de Operaciones Lógicas (&&, ||)

Exige estrictamente operandos de tipo bool.

Izq / Der	numerus	decimalis	textum	littera	bool
bool	ERROR	ERROR	ERROR	ERROR	bool

1.5. Tabla de Compatibilidad de Asignación (Destino = Origen)

Destino / Origen	numerus	decimalis	textum	littera	bool
numerus	OK	OK (conversión)	ERROR	ERROR	ERROR
decimalis	OK (conversión a tipo de mayor jerarquía)	OK	ERROR	ERROR	ERROR
textum	ERROR	ERROR	OK	ERROR	ERROR
littera	ERROR	ERROR	ERROR	OK	ERROR
bool	ERROR	ERROR	ERROR	ERROR	OK

2. Lenguaje Y? (.y)

Tipos Primitivos: entero, flotante, cadena, caracter, booleano.

2.1. Tabla de Compatibilidad de Suma (+)

En Y?, el tipo caracter se convierte al tipo de mayor jerarquía.

Izq / Der	entero	flotante	cadena	caracter	booleano
entero	entero	flotante	cadena	entero	ERROR
flotante	flotante	flotante	cadena	flotante	ERROR
cadena	cadena	cadena	cadena	cadena	cadena
caracter	entero	flotante	cadena	entero	ERROR
booleano	ERROR	ERROR	cadena	ERROR	ERROR

2.2. Tabla de Operaciones Numéricas (-, *, /)

Izq / Der	entero	flotante	cadena	caracter	booleano
entero	entero	flotante	ERROR	entero	ERROR
flotante	flotante	flotante	ERROR	flotante	ERROR
cadena	ERROR	ERROR	ERROR	ERROR	ERROR
caracter	entero	flotante	ERROR	entero	ERROR
booleano	ERROR	ERROR	ERROR	ERROR	ERROR

2.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=)

Izq / Der	entero	flotante	cadena	caracter	booleano
entero	booleano	booleano	ERROR	booleano	ERROR
flotante	booleano	booleano	ERROR	booleano	ERROR
cadena	ERROR	ERROR	booleano	ERROR	ERROR
caracter	booleano	booleano	ERROR	booleano	ERROR
booleano	ERROR	ERROR	ERROR	ERROR	booleano

2.4. Asignación (Destino = Origen)

Destino / Origen	entero	flotante	cadena	caracter	booleano
entero	OK	ERROR	ERROR	OK (conversión)	ERROR
flotante	OK (conversión a tipo de mayor jerarquía)	OK	ERROR	OK (conversión a tipo de mayor jerarquía)	ERROR
cadena	ERROR	ERROR	OK	ERROR	ERROR
caracter	ERROR	ERROR	ERROR	OK	ERROR
booleano	ERROR	ERROR	ERROR	ERROR	OK

3. Lenguaje Zetariano (.z)

Tipos Primitivos: int, double, char, boolean, String, null.

3.1. Tabla Aritmética de Suma (+)

Izq / Der	int	double	char	boolean	String
int	int	double	int	ERROR	String
double	double	double	double	ERROR	String
char	int	double	int	ERROR	String
boolean	ERROR	ERROR	ERROR	ERROR	String
String	String	String	String	String	String

3.2. Tabla de Operaciones Numéricas(-, *, /, %)

Izq / Der	int	double	char	boolean	String
int	int	double	int	ERROR	ERROR
double	double	double	double	ERROR	ERROR
char	int	double	int	ERROR	ERROR
boolean	ERROR	ERROR	ERROR	ERROR	ERROR
String	ERROR	ERROR	ERROR	ERROR	ERROR

3.3. Tabla de Operaciones Relacionales (==, !=, >, <, >=, <=)

En Zetariano, la constante null es válida en comparaciones contra cualquier referencia u objeto de clase.

Izq / Der	int	double	char	boolean	String	null
int	boolean	boolean	boolean	ERROR	ERROR	boolean
double	boolean	boolean	boolean	ERROR	ERROR	boolean
char	boolean	boolean	boolean	ERROR	ERROR	boolean
boolean	ERROR	ERROR	ERROR	boolean	ERROR	boolean
String	ERROR	ERROR	ERROR	ERROR	boolean	boolean
null	boolean	boolean	boolean	boolean	boolean	boolean

3.4. Asignación (Destino = Origen)

Destino / Origen	int	double	char	boolean	String	Clase X	null
int	OK	ERROR	OK (conversión)	ERROR	ERROR	ERROR	OK
double	OK (conversión a tipo de mayor jerarquía)	OK	ERROR	ERROR	ERROR	ERROR	OK
char	ERROR	ERROR	OK	ERROR	ERROR	ERROR	OK
boolean	ERROR	ERROR	ERROR	OK	ERROR	ERROR	OK
String	ERROR	ERROR	ERROR	ERROR	OK	ERROR	OK
Clase X	ERROR	ERROR	ERROR	ERROR	ERROR	OK (misma clase)	OK

3.5. Operador Ternario (cond ? RamaVerdadero : RamaFalso)

RamaV / RamaF	int	double	char	boolean	String
int	int	double	int	ERROR	ERROR
double	double	double	double	ERROR	ERROR
char	int	double	char	ERROR	ERROR
boolean	ERROR	ERROR	ERROR	boolean	ERROR
String	ERROR	ERROR	ERROR	ERROR	String

4. Mapeo Unificado de Tipos a Código C Nativo

Todos los tipos analizados semánticamente en el motor de compilación se traducen a C nativo según la siguiente tabla de equivalencias usadas en el método `mapearTipo` en `ContextoTraduccion`:

Tipo Pig Latin	Tipo Lenguaje Y?	Tipo Zetariano	Tipo Generado en C	Descripción en C
numerus	entero	int	int	Entero con signo de 32 bits.
decimalis	flotante	double	double	Coma flotante de doble precisión.
textum	cadena	String	char*	Puntero a cadena reservado dinámicamente en Heap.
littera	caracter	char	char	Carácter ASCII.
bool	booleano	boolean	bool	Booleano nativo via <code><stdbool.h></code> .
(sin tipo)	(sin tipo)	void	void	Retorno nulo en funciones o métodos.
Estructura	estructura	class Clase	struct X / struct X*	Estructuras Y por valor, clases Zetariano en Heap.
series T[]	T[]	T[]	T* / T[]	Arreglo aplanado en memoria.